

Aléas possibles pour le sujet V2

1. Plafond journalier par client

Demande possible du prof

Un client ne peut pas transférer plus de 2 000 000 Ar par jour.

Idée

On additionne tous les transferts faits aujourd'hui par le compte source.

Exemple de requête :

```
SELECT SUM(montant)
FROM transactions
WHERE compte_source_id = 1
AND type_operation_id = 3
AND DATE(date_transaction) = '2026-07-21';
```

Dans `TransactionModel.php`

Créer une fonction :

```
public function getTotalTransfertJour($compteId)
{
    return $this
        ->selectSum('montant', 'total')
        ->where('compte_source_id', $compteId)
        ->where('type_operation_id', 3)
        ->where('DATE(date_transaction)', date('Y-m-d'))
        ->first();
}
```

Dans `TransactionController.php`

Créer une fonction :

```
private function verifierPlafondJournalier($transactionModel, $compteId, $montant)
{
    $resultat = $transactionModel->getTotalTransfertJour($compteId);
    $totalJour = $resultat['total'] ?? 0;

    if ($totalJour + $montant > 2000000) {
        return 'Tu as atteint le montant maximum de transfert pour aujourd
```

```
hui. Reessaie demain.';
    }

    return null;
}
```

Puis dans `enregistrerTransfert()`, avant d'enregistrer :

```
$erreurPlafond = $this->verifierPlafondJournalier($transactionModel,
$source['id'], $montant);

if ($erreurPlafond) {
    return $this->retourErreur($erreurPlafond);
}
```

Dans la vue

Si la vue affiche déjà les erreurs avec `session()->getFlashdata( 'error' )`, il n'y a rien d'autre à faire.

2. Transfert multiple avec montant different par destinataire

Demande possible du prof

Au lieu de diviser un montant total, chaque destinataire doit avoir son propre montant.

Idée

Dans la vue, chaque ligne aura :

- un numero
- un montant

Exemple :

```
<input type="text" name="telephones[]" placeholder="Numero">
<input type="number" name="montants[]" placeholder="Montant">
```

Dans `TransactionController.php`

On recupere les deux tableaux :

```
$telephones = $this->request->getPost( 'telephones' );
$montants = $this->request->getPost( 'montants' );
```

Puis on parcourt :

```
foreach ($telephones as $index => $telephone) {  
    $montant = $montants[$index];  
  
    if ($montant <= 0) {  
        return $this->retourErreur('Montant invalide pour un  
destinataire');  
    }  
}
```

Ce qu'il faut changer

Il faut supprimer la logique qui fait :

```
$montantParDestinataire = $montantTotal / $nombreDestinataires;
```

Et utiliser le montant de chaque ligne.

3. Annuler une transaction

Demande possible du prof

L'opérateur peut annuler une transaction.

Idée

On ajoute un statut :

```
SUCCES  
ANNULE
```

Si on annule :

- on remet l'argent au compte source
- on retire l'argent du compte destination
- on met la transaction en **ANNULE**

Dans **TransactionController.php**

Créer une fonction :

```
public function annulerTransaction($id)  
{  
    $transactionModel = new TransactionModel();  
    $compteModel = new CompteModel();
```

```

    $transaction = $transactionModel->find($id);

    if (!$transaction) {
        return redirect()->back()->with('error', 'Transaction introuvable');
    }

    if ($transaction['statut'] == 'ANNULE') {
        return redirect()->back()->with('error', 'Transaction deja annulee');
    }

    $source = $compteModel->find($transaction['compte_source_id']);
    $destination = $compteModel->find($transaction['compte_destination_id']);

    $totalDebite = $transaction['montant'] + $transaction['frais'] + $transaction['commission'];

    $compteModel->update($source['id'], [
        'solde' => $source['solde'] + $totalDebite
    ]);

    $compteModel->update($destination['id'], [
        'solde' => $destination['solde'] - $transaction['montant_recu']
    ]);

    $transactionModel->update($id, [
        'statut' => 'ANNULE'
    ]);

    return redirect()->back()->with('success', 'Transaction annulee');
}

```

Attention

Pour un depot ou un retrait, la logique d'annulation n'est pas exactement la meme. Donc il faut verifier le `type_operation_id`.

4. Commission differente selon le montant

Demande possible du prof

Orange prend 2% jusqu'a 500 000 Ar, puis 3% au-dessus.

Idée

La table actuelle `commissions_inter_operateurs` contient seulement :

```
autre_operateur_id
```

pourcentage

Il faut ajouter des colonnes :

montant_min
montant_max

Exemple de nouvelle table

```
CREATE TABLE commissions_inter_operateurs (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    autre_operateur_id INTEGER NOT NULL,  
    montant_min INTEGER NOT NULL,  
    montant_max INTEGER NOT NULL,  
    pourcentage REAL NOT NULL  
);
```

Dans `CommissionInterOperateurModel.php`

Créer une fonction :

```
public function getCommissionByMontant($autreOperateurId, $montant)  
{  
    return $this  
        ->where('autre_operateur_id', $autreOperateurId)  
        ->where('montant_min <=', $montant)  
        ->where('montant_max >=', $montant)  
        ->first();  
}
```

Dans `TransactionController.php`

Dans la fonction qui calcule la commission, remplacer la recherche simple par :

```
$commissionData = $commissionModel-  
>getCommissionByMontant($autreOperateurId, $montant);
```

5. Solde minimum obligatoire

Demande possible du prof

Après un transfert ou un retrait, le compte doit toujours garder au moins **1 000 Ar.**

Idée

Avant de debiter, on verifie :

```
$soldeAprèsOperation = $compte['solde'] - $total;
```

Si le solde restant est inferieur a 1000, on refuse.

Dans `TransactionController.php`

Créer une fonction :

```
private function verifierSoldeMinimum($compte, $total)
{
    $soldeMinimum = 1000;
    $soldeRestant = $compte['solde'] - $total;

    if ($soldeRestant < $soldeMinimum) {
        return 'Operation refusee. Il faut garder au moins 1 000 Ar sur le
compte.';
    }

    return null;
}
```

Puis dans `enregistrerTransfert()` ou `enregistrerRetrait()` :

```
$erreurSoldeMinimum = $this->verifierSoldeMinimum($source,
$couts['total']);

if ($erreurSoldeMinimum) {
    return $this->retourErreur($erreurSoldeMinimum);
}
```

Pour le retrait, utiliser :

```
$erreurSoldeMinimum = $this->verifierSoldeMinimum($compte, $total);
```

6. Heure limite pour faire un transfert

Demande possible du prof

Les transferts sont autorises seulement entre 06:00 et 22:00.

Idee

Avant d'enregistrer le transfert, on verifie l'heure actuelle.

Dans `TransactionController.php`

Créer une fonction :

```
private function verifierHeureTransfert()
{
    $heure = date('H:i');

    if ($heure < '06:00' || $heure > '22:00') {
        return 'Les transferts sont disponibles seulement entre 06:00 et 22:00.';
    }

    return null;
}
```

Puis dans `enregistrerTransfert()` :

```
$erreurHeure = $this->verifierHeureTransfert();

if ($erreurHeure) {
    return $this->retourErreur($erreurHeure);
}
```

On peut faire pareil dans `enregistrerTransfertMultiple()`.

7. Nombre maximum de destinataires

Demande possible du prof

Dans un transfert multiple, on ne peut pas envoyer a plus de 5 destinataires.

Idee

On compte les numeros dans le tableau `$telephones`.

Dans `TransactionController.php`

Dans la fonction de validation du transfert multiple, ajouter :

```
if (count($telephones) > 5) {
    return 'Vous ne pouvez pas envoyer a plus de 5 destinataires.';
}
```

Dans la vue

On peut aussi bloquer le bouton + **Ajouter** quand il y a déjà 5 lignes.

Exemple :

```
if (document.querySelectorAll('.destinataire-ligne').length >= 5) {  
    alert('Maximum 5 destinataires');  
    return;  
}
```

8. Frais réduits pour les gros montants

Demande possible du prof

Si le montant est supérieur à **1 000 000 Ar**, on réduit les frais de transfert de **50%**.

Idee

On calcule d'abord le frais normal, puis on applique la réduction.

Dans **TransactionController.php**

Après avoir trouvé **\$fraisTransfert** :

```
if ($montant > 1000000) {  
    $fraisTransfert = $fraisTransfert / 2;  
}
```

Variante plus propre

Créer une petite fonction :

```
private function appliquerReductionGrosMontant($montant, $frais)  
{  
    if ($montant > 1000000) {  
        return $frais / 2;  
    }  
  
    return $frais;  
}
```

Puis utiliser :

```
$fraisTransfert = $this->appliquerReductionGrosMontant($montant,  
$fraisTransfert);
```

9. Bloquer un compte client

Demande possible du prof

Si le compte est **BLOQUE**, le client ne peut plus faire de depot, retrait ou transfert.

Idee

La table **comptes** a deja une colonne **statut**.

On verifie le statut avant chaque operation.

Dans **TransactionController.php**

Créer une fonction :

```
private function verifierCompteActif($compte)
{
    if ($compte['statut'] != 'ACTIF') {
        return 'Votre compte est bloque. Operation impossible.';
    }

    return null;
}
```

Puis dans **enregistrerTransfert()** :

```
$erreurCompte = $this->verifierCompteActif($source);

if ($erreurCompte) {
    return $this->retourErreur($erreurCompte);
}
```

Faire pareil dans **enregistrerDepot()** et **enregistrerRetrait()**.

10. Rechercher les transactions par date côté opérateur

Demande possible du prof

L'operateur veut filtrer les transactions entre deux dates.

Idee

Ajouter deux champs dans la vue :

- date debut
- date fin

Puis filtrer dans le controleur.

Dans `TransactionController.php`

Dans la fonction `index()` côté opérateur :

```
$dateDebut = $this->request->getGet('date_debut');
$dateFin = $this->request->getGet('date_fin');

$transactionModel = new TransactionModel();
$query = $transactionModel->orderBy('date_transaction', 'DESC');

if ($dateDebut != '') {
    $query->where('DATE(date_transaction) >=', $dateDebut);
}

if ($dateFin != '') {
    $query->where('DATE(date_transaction) <=', $dateFin);
}

$data = [
    'title' => 'Liste des transactions',
    'transactions' => $query->findAll(),
];
```

Dans la vue `transaction/all.php`

Ajouter un formulaire en haut :

```
<form method="get">
    <input type="date" name="date_debut">
    <input type="date" name="date_fin">
    <button type="submit">Filtrer</button>
</form>
```

11. Frais gratuits entre deux clients du même opérateur

Demande possible du prof

Si le transfert est entre deux numéros de notre opérateur, les frais de transfert sont gratuits.

Idee

Si `$autreOperateurId` est `null`, alors `$fraisTransfert = 0`.

Dans `TransactionController.php`

Dans la fonction `calculerCoutsTransfert()` :

```
if ($autreOperateurId === null) {  
    $fraisTransfert = 0;  
} else {  
    $baremeTransfert = $baremeModel->getFraisByMontant(3, $montant, null);  
  
    if (!$baremeTransfert) {  
        return ['erreur' => 'Bareme de transfert introuvable'];  
    }  
  
    $fraisTransfert = $baremeTransfert['frais'];  
}
```

Attention

Si le prof veut aussi garder les frais de retrait, il ne faut pas les supprimer. On met seulement les frais de transfert a 0.

12. Code secret avant validation d'une operation

Demande possible du prof

Avant de confirmer un transfert ou un retrait, le client doit entrer un code secret.

Idee

Ajouter un champ `code_secret` pour le client, puis comparer avec le code envoye dans le formulaire.

Dans la table `clients`

Ajouter une colonne :

```
ALTER TABLE clients ADD COLUMN code_secret TEXT;
```

Dans la vue

Ajouter un input :

```
<input type="password" name="code_secret" placeholder="Code secret">
```

Dans `TransactionController.php`

Avant d'enregistrer :

```
$codeSecret = $this->request->getPost('code_secret');
```

```
if ($codeSecret != $client['code_secret']) {  
    return $this->retourErreur('Code secret incorrect');  
}
```

Attention

Pour faire mieux, il faudrait hasher le code secret. Mais si le prof demande vite, une comparaison simple peut suffire pour montrer la logique.

13. Refuser les montants avec des centimes

Demande possible du prof

Les montants doivent etre des nombres entiers seulement.

Idee

Un montant comme 1000.50 doit etre refuse.

Dans `TransactionController.php`

Créer une fonction :

```
private function verifierMontantEntier($montant)  
{  
    if ($montant != floor($montant)) {  
        return 'Le montant doit etre un nombre entier.';  
    }  
  
    return null;  
}
```

Puis dans `enregistrerTransfert()` :

```
$erreurMontant = $this->verifierMontantEntier($montant);  
  
if ($erreurMontant) {  
    return $this->retourErreur($erreurMontant);  
}
```

Dans la vue

Mettre :

```
<input type="number" step="1" name="montant">
```

14. Limiter le nombre de transferts par jour

Demande possible du prof

Un client ne peut faire que 3 transferts par jour.

Idee

On compte les transactions de type transfert faites aujourd'hui par le compte source.

Dans `TransactionModel.php`

Créer une fonction :

```
public function compterTransfertsJour($compteId)
{
    return $this
        ->where('compte_source_id', $compteId)
        ->where('type_operation_id', 3)
        ->where('DATE(date_transaction)', date('Y-m-d'))
        ->countAllResults();
}
```

Dans `TransactionController.php`

Créer une fonction :

```
private function verifierNombreTransfertsJour($transactionModel, $compteId)
{
    $nombre = $transactionModel->compterTransfertsJour($compteId);

    if ($nombre >= 3) {
        return 'Vous avez deja fait 3 transferts aujourd'hui.';
    }

    return null;
}
```

Puis dans `enregistrerTransfert()` :

```
$erreurNombre = $this->verifierNombreTransfertsJour($transactionModel,
$source['id']);

if ($erreurNombre) {
    return $this->retourErreur($erreurNombre);
}
```

15. Justificatif obligatoire pour gros transfert

Demande possible du prof

Si le montant dépasse 1 000 000 Ar, le client doit saisir une raison.

Idee

Ajouter un champ `motif` dans le formulaire.

Dans la table `transactions`

Ajouter une colonne :

```
ALTER TABLE transactions ADD COLUMN motif TEXT;
```

Dans la vue

Ajouter :

```
<textarea name="motif" placeholder="Motif du transfert"></textarea>
```

Dans `TransactionController.php`

Avant l'enregistrement :

```
$motif = $this->request->getPost('motif');

if ($montant > 1000000 && trim($motif) == '') {
    return $this->retourErreur('Le motif est obligatoire pour un gros transfert.');
```

Puis il faut enregistrer `$motif` dans la transaction.

16. Exporter la liste des transactions en CSV

Demande possible du prof

L'opérateur veut exporter les transactions en fichier CSV.

Idee

Créer une route `/opérateur/transactions/export`.

Dans `Routes.php`

Ajouter :

```
$routes->get('transactions/export', 'TransactionController::exportCsv');
```

Dans `TransactionController.php`

Créer une fonction :

```
public function exportCsv()
{
    $transactionModel = new TransactionModel();
    $transactions = $transactionModel->findAll();

    $contenu = "ID,Type,Montant,Frais,Commission,Date\n";

    foreach ($transactions as $transaction) {
        $contenu .= $transaction['id'] . ",";
        $contenu .= $transaction['type_operation_id'] . ",";
        $contenu .= $transaction['montant'] . ",";
        $contenu .= $transaction['frais'] . ",";
        $contenu .= $transaction['commission'] . ",";
        $contenu .= $transaction['date_transaction'] . "\n";
    }

    return $this->response
        ->setHeader('Content-Type', 'text/csv')
        ->setHeader('Content-Disposition', 'attachment;
filename=transactions.csv')
        ->setBody($contenu);
}
```

Dans la vue

Ajouter un bouton :

```
<a href="<?= base_url('opérateur/transactions/export') ?>">Exporter CSV</a>
```

17. Historique avec filtre par type d'operation

Demande possible du prof

Le client veut voir seulement les depots, les retraits ou les transferts.

Idee

Ajouter un select dans l'historique client.

Dans la vue `historique.php`

Ajouter :

```
<form method="get">
  <select name="type_operation_id">
    <option value="">Tous</option>
    <option value="1">Depot</option>
    <option value="2">Retrait</option>
    <option value="3">Transfert</option>
  </select>
  <button type="submit">Filtrer</button>
</form>
```

Dans `TransactionModel.php`

Modifier ou créer une fonction :

```
public function getHistoriqueFiltre($compteId, $typeOperationId)
{
    $query = $this
        ->select('transactions.*, types_operations.libelle AS
type_operation')
        ->join('types_operations', 'types_operations.id =
transactions.type_operation_id')
        ->groupStart()
        ->where('compte_source_id', $compteId)
        ->orWhere('compte_destination_id', $compteId)
        ->groupEnd();

    if ($typeOperationId != '') {
        $query->where('type_operation_id', $typeOperationId);
    }

    return $query->orderBy('date_transaction', 'DESC')->findAll();
}
```

18. Recevoir une notification apres transaction

Demande possible du prof

Apres un depot, retrait ou transfert, on doit enregistrer une notification pour le client.

Idee

Créer une table `notifications`.

Dans `base.sql`

Ajouter :

```
CREATE TABLE notifications (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    client_id INTEGER NOT NULL,  
    message TEXT NOT NULL,  
    lu INTEGER NOT NULL DEFAULT 0,  
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

Dans `TransactionController.php`

Après une transaction réussie :

```
db_connect()->table('notifications')->insert([  
    'client_id' => $clientId,  
    'message' => 'Votre transfert a ete effectue avec succes',  
    'lu' => 0  
]);
```

Dans l'accueil

On peut afficher le nombre de notifications non lues a cote de l'icone cloche.

19. Verifier que le numero a exactement 10 chiffres

Demande possible du prof

Le numero doit contenir exactement 10 chiffres, pas de lettres.

Idee

Utiliser `preg_match`.

Dans `TransactionController.php`

Créer une fonction :

```
private function verifierNumero($telephone)  
{  
    if (!preg_match('/^[0-9]{10}$/', $telephone)) {  
        return 'Le numero doit contenir exactement 10 chiffres.';  
    }  
  
    return null;  
}
```

Puis dans `enregistrerTransfert()` :

```
$erreurNumero = $this->verifierNumero($telephone);  
  
if ($erreurNumero) {  
    return $this->retourErreur($erreurNumero);  
}
```

20. Afficher le gain net de l'operateur

Demande possible du prof

Dans la situation des gains, afficher le gain net :

```
gain net = frais encaisses - commissions a payer aux autres operateurs
```

Idee

On additionne les frais et on soustrait les commissions.

Dans `GainModel.php`

Créer une fonction :

```
public function getGainNet()  
{  
    return $this  
        ->select('SUM(frais) - SUM(commission) AS gain_net')  
        ->first();  
}
```

Dans `GainController.php`

Ajouter :

```
$data['gainNet'] = $model->getGainNet();
```

Dans la vue `gains/index.php`

Afficher :

```
<?= number_format($gainNet['gain_net'] ?? 0, 0, ',', ' ') ?> Ar
```

Resume rapide

Les aléas possibles sont :

1. plafond journalier
2. montant different par destinataire
3. annulation de transaction
4. commission selon tranche de montant
5. solde minimum obligatoire
6. heure limite pour faire un transfert
7. nombre maximum de destinataires
8. frais reduits pour gros montant
9. compte client bloque
10. recherche des transactions par date
11. frais gratuits entre clients internes
12. code secret obligatoire
13. montant entier seulement
14. nombre de transferts limite par jour
15. motif obligatoire pour gros transfert
16. export CSV des transactions
17. filtre historique par type
18. notifications apres transaction
19. numero exactement 10 chiffres
20. gain net de l'operateur

Dans presque tous les cas, il faut faire :

1. une fonction dans le Model pour chercher les donnees
2. une fonction simple dans le Controller pour verifier
3. un `if` avant l'enregistrement
4. un message d'erreur dans la vue